5. Autoregressive Model

Young-geun Kim

Department of Mathematical Sciences

FS26 MTH345: Deep Generative Models: A Statistical Perspective





Lecture Topics

M1. Foundations of Probabilistic Modelling

1. Fundamentals of Probability Theory
2. Random Variables
3. Likelihood Maximization Principle
4. Basics of Probabilistic Modelling

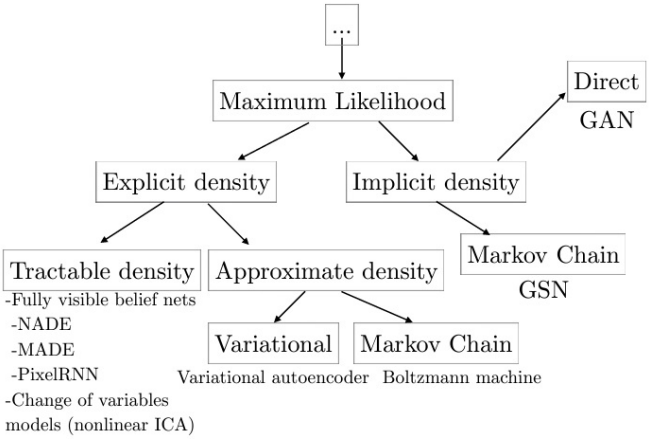
M2. Likelihood Ratio-based Generative Models

5. Autoregressive Model
6. Energy-based Model
7. Variational Autoencoder
8. Generative Adversarial Network

M3. Generative Models with Other Statistical Distance

9. Integral Probability Metric-based Approaches
10. Wasserstein Distance-based Approaches
11. Score-based Approaches

Lecture Topics



- The goal of the second module is to provide a comprehensive understanding of generative models based on maximum likelihood estimation.

The figure is from Goodfellow (2016).

Lecture Topics

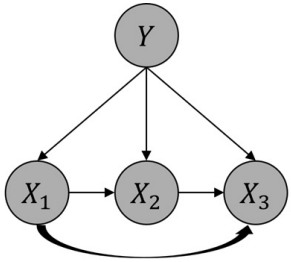
- In this chapter, we begin with traditional autoregressive models, and transition into deep learning-based approaches.
- Recommended reference materials include:
  - Uria et al. (2016), "Neural Autoregressive Distribution Estimation," *JMLR*.
  - Goodfellow (2016), "Nips 2016 tutorial: Generative adversarial networks," *arXiv*.

Table of Contents

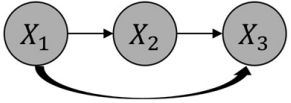
1. Autoregressive Model

2. Deep Learning-based Approaches

Unconditional Generative Model



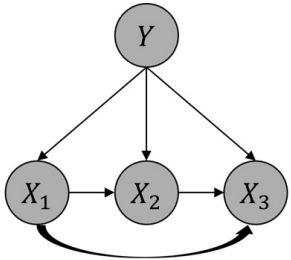
(a) Conditional Generative Model



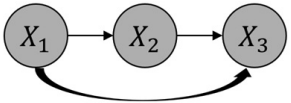
(b) (Unconditional) Generative Model

- In the previous chapter, we utilized labels (Y) to model the conditional distribution of observed feature vectors ($\boldsymbol{X} = (X_1, \ldots, X_d)^T$) given labels.
- However, labels are usually scarce and expensive to obtain, which motivates the need for unconditional (or *unsupervised*) approaches.

Unconditional Generative Model



(a) Conditional Generative Model



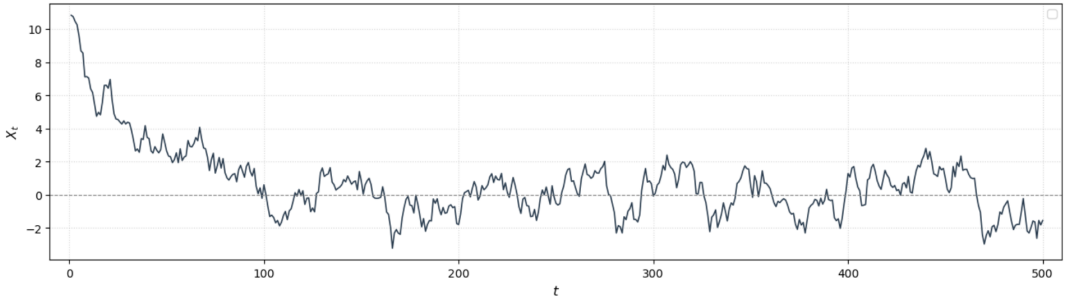
(b) (Unconditional) Generative Model

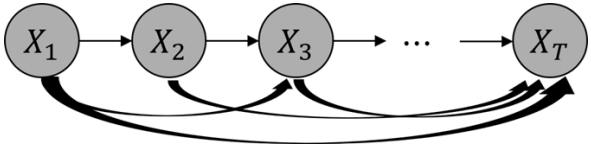
- *Autoregressive models*, which mean self-regression models, learn conditional distributions where each feature is regressed on its preceding variables:

$$f(\boldsymbol{x}) = f(x_1)f(x_2 \mid x_1) \cdots f(x_d \mid x_1, \ldots, x_{d-1}) \tag{1}$$

- Depending on domain knowledge, each individual conditional distribution can be parameterized as $f_\theta(x_i \mid x_1, \ldots, x_{i-1})$.

Autoregressive Model





Autoregressive Model

- With only a single observed trajectory $\boldsymbol{X} = (X_1, \ldots, X_T)^T$, we need further structural assumptions to learn the joint distribution.
- **Stationarity:**

$$f(x_t \mid x_{t-1}, \ldots, x_{t-p}) = f(x_{t'} \mid x_{t'-1}, \ldots, x_{t'-p}) \quad \forall t, t' \tag{2}$$

The conditional distribution remains invariant over time, allowing us to pool sequential transitions as repeated samples in a slided design matrix (when $p = 2$):

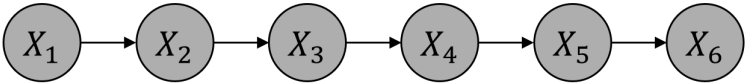
Row i	**Base** t	X_{t-2}	X_{t-1}	X_t
1	$t = 3$	10.8274	10.7400	10.4549
2	$t = 4$	10.7400	10.4549	10.2577
3	$t = 5$	10.4549	10.2577	9.5852
⋮	⋮	⋮	⋮	⋮
498	$t = 500$	-1.5419	-1.8138	-1.5440

Autoregressive Model

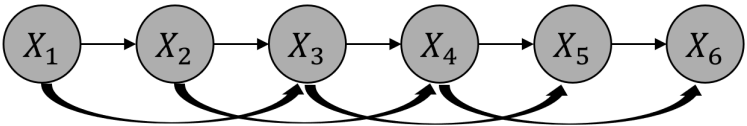
- **Markov Property (Order p):**

$$f(x_t \mid x_1, \ldots, x_{t-1}) = f(x_t \mid x_{t-1}, \ldots, x_{t-p}) \tag{3}$$

Given the recent p observations, earlier ones provide no additional information about the current datum X_t.



(a) Markov property (Order 1)



(b) Markov property (Order 2)

Autoregressive Model

- Autoregressive model of order p (AR(p)) models the data generation mechanism as follows, assuming both stationarity and the Markov property of order p:

$$
\begin{aligned}
(X_1, \ldots, X_p)^T &\sim f(x_1, \ldots, x_p) \\
X_{p+1} &= \sum_{j=1}^{p} \rho_j X_{p+1-j} + \epsilon_{p+1} \\
&\vdots \\
X_T &= \sum_{j=1}^{p} \rho_j X_{T-j} + \epsilon_T
\end{aligned}
\tag{4}
$$

  where $\epsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$ for $t = p+1, \ldots, T$.

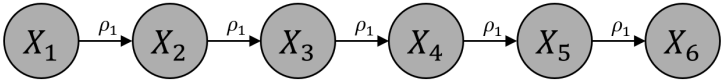
- This formulation highlights that the model generates observations sequentially.

Autoregressive Model

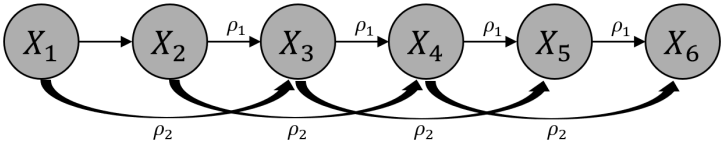
- The joint distribution specified by AR(p) can be formulated as:

$$f_\theta(\boldsymbol{x}) = f(x_1, \ldots, x_p) \prod_{t=p+1}^{T} f_\theta(x_t \mid x_{t-1}, \ldots, x_{t-p}) \tag{5}$$

where $\theta = (\rho_1, \ldots, \rho_p, \sigma^2)^T$.



(a) Autoregressive Model of Order 1



(b) Autoregressive Model of Order 2

Autoregressive Model

- Here, we focus on deriving the explicit density functions[1] and obtaining the MLE when the model order is $p = 1$:

$$\begin{aligned} f_\theta(\boldsymbol{x}) &= f(x_1) \prod_{t=2}^{T} f_\theta(x_t \mid x_{t-1}) \\ &= f(x_1) \prod_{t=2}^{T} \left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_t - \rho_1 x_{t-1})^2}{2\sigma^2} \right) \right) \end{aligned} \tag{6}$$

- The corresponding log-likelihood can be formulated as:

$$\log f_\theta(\boldsymbol{x}) = \log f(x_1) - \frac{T-1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{t=2}^{T} (x_t - \rho_1 x_{t-1})^2 \tag{7}$$

[1] Here, *explicit density* means that the joint distribution is analytical.

Autoregressive Model

Exercise 1

Let $X_1, \ldots, X_T$ follow an AR(1) process:

$$X_t = \rho_1 X_{t-1} + \epsilon_t, \quad t = 2, \ldots, T, \tag{8}$$

where $\epsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$. Find the (conditional) MLE of $\theta = (\rho_1, \sigma^2)^\top$.

Autoregressive Model



Autoregressive Models

- The parameter estimation results for each model order p are as follows:

$$\begin{aligned}
\text{AR(1)}: \quad & X_t = 0.9411 X_{t-1} && + e_t \\
\text{AR(2)}: \quad & X_t = 0.9555 X_{t-1} - 0.0145 X_{t-2} && + e_t \\
\text{AR(3)}: \quad & X_t = 0.9556 X_{t-1} - 0.0263 X_{t-2} + 0.0117 X_{t-3} && + e_t \\
\text{AR(4)}: \quad & X_t = 0.9560 X_{t-1} - 0.0270 X_{t-2} + 0.0354 X_{t-3} - 0.0235 X_{t-4} && + e_t
\end{aligned} \tag{9}$$

- The true data generation structure formulas used to simulate this time series are:

$$\begin{aligned}
& X_1 \sim \mathcal{N}(0, 16) \\
& X_t = 0.95 X_{t-1} + \epsilon_t, \quad \epsilon_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 0.25) \quad \text{for } t = 2, \ldots, T
\end{aligned} \tag{10}$$

Autoregressive Models

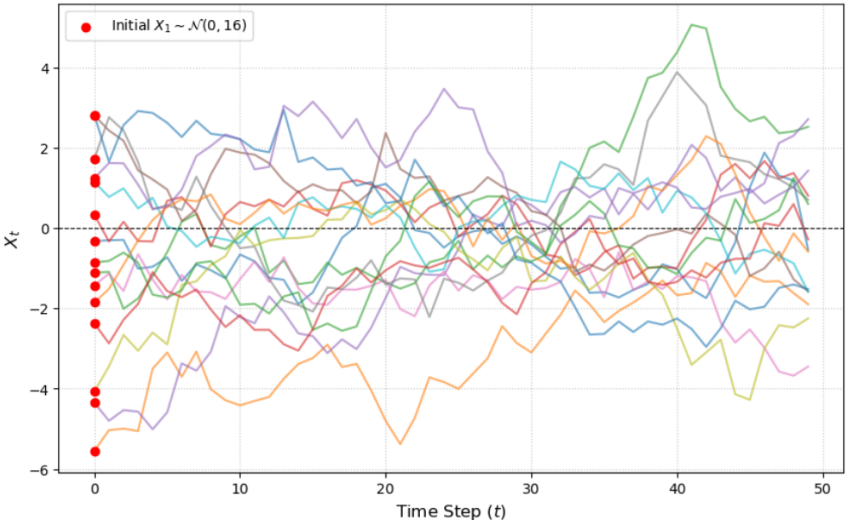


Table of Contents

1. Autoregressive Model

2. Deep Learning-based Approaches

Fully Visible Sigmoid Belief Network

- When data are complex and high-dimensional, the aforementioned probabilistic approaches struggle to capture the joint distribution.
- In particular, standard Markovian assumptions and global stationarity fail to account for the grid-like *spatial dependencies* in structured data such as images.
- Here, we focus on 28×28 MNIST data, $\boldsymbol{X} = (X_1, \ldots, X_{784})^T$, where $X_i \in \{0, 1\}$.

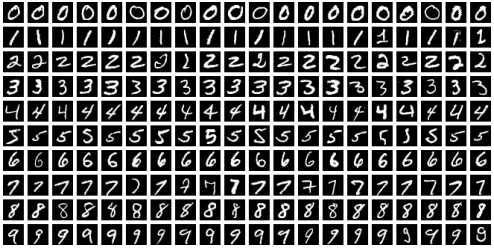
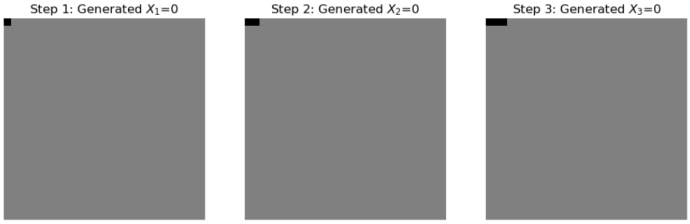


Image source: `https://en.wikipedia.org/wiki/MNIST_database`.

Fully Visible Sigmoid Belief Network

x_1	x_2	x_3	$p(x_2 \mid x_1)$	$p(x_3 \mid x_1, x_2)$
0	0	0	1.0000	1.0000
0	0	1	1.0000	0.0000
0	1	0	0.0000	0.0000
0	1	1	0.0000	0.0000
1	0	0	0.0000	0.0000
1	0	1	0.0000	0.0000
1	1	0	0.0000	0.0000
1	1	1	0.0000	0.0000



- Sequential sampling: $X_t \sim p(x_t \mid x_{<t})$.
- **Q**: What is the size of the conditional probability table for $p(X_{784} \mid X_1, \ldots, X_{783})$?

Fully Visible Sigmoid Belief Network

- The Fully Visible Sigmoid Belief Network (Neal, 1992)[2] is a logistic autoregressive model:

$$p_\theta(x_j = 1 \mid x_1, x_2, \ldots, x_{j-1}) = \mathrm{sig}(\alpha^{(j)} + \boldsymbol{\beta}^{(j)\,T}(x_1, \ldots, x_{j-1})^T) \tag{11}$$

  where $\mathrm{sig}(x) := 1/(1 + \exp(-x))$ represents the sigmoid function and $\theta := (\alpha^{(1)}, \ldots, \alpha^{(d)}, \boldsymbol{\beta}^{(2)}, \ldots, \boldsymbol{\beta}^{(d)})^T$.
- Note that none of the $784 - 1 = 783$ logistic regression models share any parameters, yielding excessive number of parameters ($O(d^2)$).

[2] The name is derived from the Boltzmann machine literature and will be discussed in detail in the next chapter on energy-based models.

Fully Visible Sigmoid Belief Network

Exercise 2

Consider a Fully Visible Sigmoid Belief Network (FVSBN) for $\boldsymbol{X} \in \{0,1\}^d$:

$$X_j \mid \boldsymbol{X}_{<j} = \boldsymbol{x}_{<j} \sim \text{Ber}\left(\mu_\theta^{(j)}(\boldsymbol{x}_{<j})\right), \quad j = 1, \ldots, d, \tag{12}$$

where $\mu_\theta^{(j)}(\boldsymbol{x}_{<j}) = \text{sig}\left(\alpha^{(j)} + (\boldsymbol{\beta}^{(j)})^\top \boldsymbol{x}_{<j}\right)$ for $j \geq 2$, and $\mu_\theta^{(1)} = \text{sig}(\alpha^{(1)})$.

Fully Visible Sigmoid Belief Network

1. Show that the log-likelihood of a single sample $\boldsymbol{x} \in \{0,1\}^d$ decomposes into a sum of d binary cross-entropy terms:

$$\log p_\theta(\boldsymbol{x}) = \sum_{j=1}^{d} \left(x_j \log \mu_\theta^{(j)}(\boldsymbol{x}_{<j}) + (1 - x_j) \log \left(1 - \mu_\theta^{(j)}(\boldsymbol{x}_{<j}) \right) \right). \tag{13}$$

Fully Visible Sigmoid Belief Network

2. Derive the gradient of $\log p_\theta(\boldsymbol{x})$ with respect to the weight vector $\boldsymbol{\beta}^{(j)}$ for any $j \in \{2, \ldots, d\}$.

Fully Visible Sigmoid Belief Network

- The data can be generated in a sequential manner:

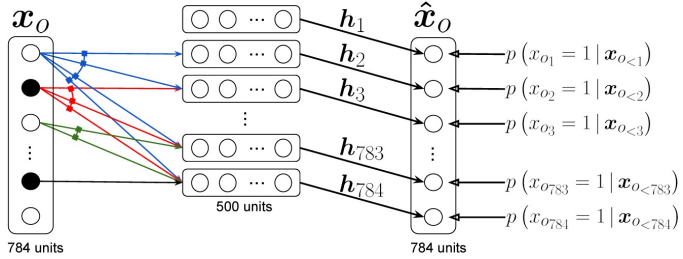
1. $\tilde{x}_1 \sim \text{Ber}\left(\text{sig}(\hat{\alpha}_n^{(1)})\right)$
2. $\tilde{x}_2 \sim \text{Ber}\left(\text{sig}\left(\hat{\alpha}_n^{(2)} + \hat{\beta}_n^{(2)}\tilde{x}_1\right)\right)$
3. $\tilde{x}_3 \sim \text{Ber}\left(\text{sig}\left(\hat{\alpha}_n^{(3)} + (\hat{\boldsymbol{\beta}}_n^{(3)})^\top(\tilde{x}_1, \tilde{x}_2)^\top\right)\right)$

$\vdots$

d. $\tilde{x}_d \sim \text{Ber}\left(\text{sig}\left(\hat{\alpha}_n^{(d)} + (\hat{\boldsymbol{\beta}}_n^{(d)})^\top \tilde{\boldsymbol{x}}_{<d}\right)\right)$



Neural Autoregressive Density Estimation



- The Neural Autoregressive Density Estimation (Uria et al., 2016) introduces a neural network:

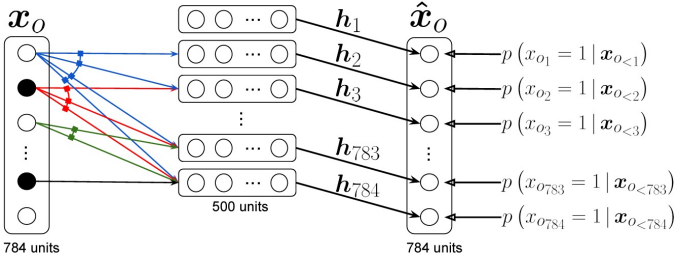
$$p_\theta(x_j = 1 \mid x_1, \ldots, x_{j-1}) = \mathrm{sig}(\boldsymbol{\alpha}_{\boldsymbol{H}}^{(j)} + W_{\boldsymbol{H}}^{(j)}(h_1^{(j)}, \ldots, h_H^{(j)})^T) \tag{14}$$

where

$$(h_1^{(j)}, \ldots, h_H^{(j)})^T = \mathrm{sig}(\boldsymbol{\alpha}_{\boldsymbol{X}}^{(j)} + W_{\boldsymbol{X}}^{(j)}(x_1, \ldots, x_{j-1})^T). \tag{15}$$

The figure is from Uria et al. (2016).

Neural Autoregressive Density Estimation



- The number of parameters in each neural network layer is $O(Hd)$, meaning that the total number of parameters across all layers scales as $O(Hd^2)$.

The figure is from Uria et al. (2016).

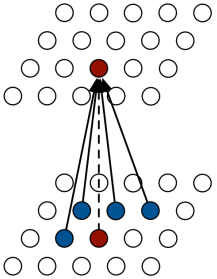
Neural Autoregressive Density Estimation



Figure 4: **Left**: samples from NADE trained on binarized MNIST. **Right**: probabilities from which each pixel was sampled. Ancestral sampling was used with the same fixed ordering used during training.

The figure is from Uria et al. (2016).

Pixel Convolutional Neural Network

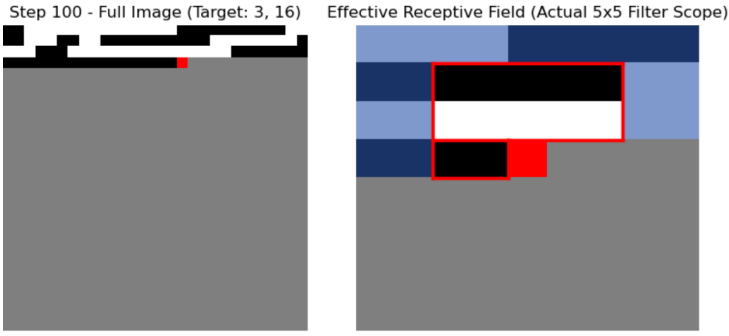


PixelCNN

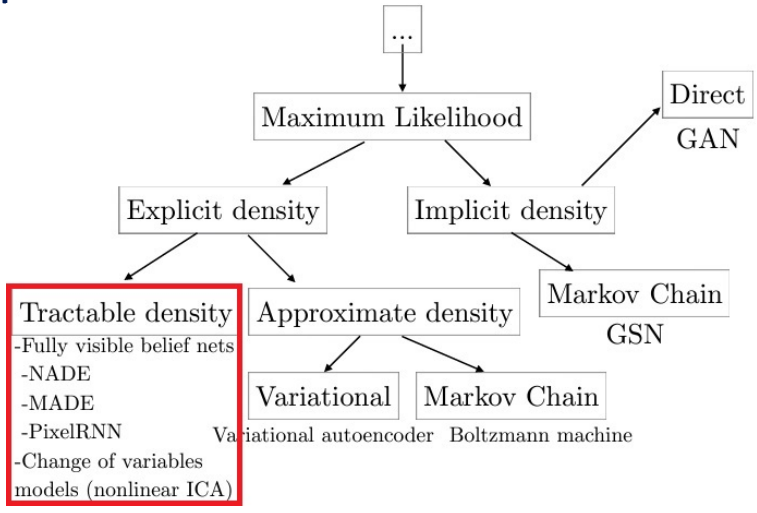
- The Pixel Convolutional Neural Network (PixelCNN) (Van den Oord et al., 2016b) utilizes spatially adjacent previous pixels in calculating conditional probabilities.
- Its variants, including Gated PixelCNN (Van den Oord et al., 2016a), perform comparably to PixelRNN (Van den Oord et al., 2016b).

The figure is from Van den Oord et al. (2016b).

Pixel Convolutional Neural Network



Lecture Topics



The figure is edited from Goodfellow (2016).

References I

Goodfellow, I. (2016). Nips 2016 tutorial: Generative adversarial networks. *arXiv preprint arXiv:1701.00160*.

Neal, R. M. (1992). Connectionist learning of belief networks. *Artificial intelligence*, 56(1):71–113.

Uria, B., Côté, M.-A., Gregor, K., Murray, I., and Larochelle, H. (2016). Neural autoregressive distribution estimation. *Journal of Machine Learning Research*, 17(205):1–37.

Van den Oord, A., Kalchbrenner, N., Espeholt, L., Vinyals, O., Graves, A., et al. (2016a). Conditional image generation with pixelcnn decoders. *Advances in neural information processing systems*, 29.

Van den Oord, A., Kalchbrenner, N., and Kavukcuoglu, K. (2016b). Pixel recurrent neural networks. In *International conference on machine learning*, pages 1747–1756. PMLR.